Lecture Notes: TwinCAT 3

Implementing a FIFO Event Buffer with TDD

March 17, 2026

Contents

1 Introduction to the FIFO Event Buffer 2

2 Utilizing FB_MemRingBuffer 2

3 The "8-Byte Discovery" (Debugging with TDD) 5

4 Integrating the Buffer via Dependency Injection 8

5 The Professional Value of TDD 13

Quick Reference Card 13

1 Introduction to the FIFO Event Buffer

To improve the event logging system, a **FIFO (First-In-First-Out)** buffer is required. This ensures that if multiple events occur rapidly, they are all captured and written to disk in the order they occurred, rather than the system only being able to handle one event at a time.

Event 1 (Oldest)	**Event 2**	**Event 3**	$\cdots$	**Event N** (Newest)
$\uparrow$ Read Head				$\uparrow$ Write Tail

Figure 1: FIFO Buffer Concept—First In, First Out

Without Buffer	**With FIFO Buffer**	**Benefit**
1 event at a time	Multiple events queued	No lost events
Events can be lost	Events preserved in order	Data integrity
Blocking behavior	Non-blocking writes	Better performance

Table 1: Comparison: Event Handling With and Without Buffer

Note: A FIFO buffer is essential in real-time systems where events may occur faster than they can be processed. The buffer acts as a temporary holding area, ensuring no data is lost during high-activity periods.

2 Utilizing FB_MemRingBuffer

The implementation relies on the `FB_MemRingBuffer` function block from the `Tc2_Utilities` **library**.

- **Purpose:** It allows writing and reading data sets of different lengths in a ring buffer.
- **Mechanism:** It operates on the FIFO principle—the oldest entries are read first.
- **Interface (Actions):** Unlike modern function blocks that use methods, `FB_MemRingBuffer` uses **Actions**. These are legacy calls that do not take arguments directly; instead, you set the block's internal variables and then trigger the action.
  - `A_AddTail`: Adds data to the end of the buffer.
  - `A_RemoveHead`: Reads and removes the oldest data set.
- **Key Variables:**
  - `pWrite` / `cbWrite`: Pointer to and size of data to be written.
  - `pBuffer` / `cbBuffer`: Pointer to and size of the memory allocated for the buffer.
  - `nCount`: Returns the current number of queued data sets.
  - `bOk`: Returns TRUE if the last action was successful.

Action/Variable	Type	Description
A_AddTail	Action	Adds data set to end of buffer
A_RemoveHead	Action	Reads and removes oldest data set
pWrite	PVOID	Pointer to data to write
cbWrite	UDINT	Size of data to write (bytes)
pRead	PVOID	Pointer to receive read data
cbRead	UDINT	Size of data to read (bytes)
pBuffer	PVOID	Pointer to allocated buffer memory
cbBuffer	UDINT	Total size of buffer memory
nCount	UDINT	Number of entries in buffer
bOk	BOOL	TRUE if last operation succeeded

Table 2: FB_MemRingBuffer Key Variables and Actions

Wrapping the Add Action

```
// ====================================================
// FUNCTION BLOCK: FB_EventBuffer
// A wrapper around FB_MemRingBuffer for storing ST_Event
// Provides a clean, method-based interface
// ====================================================

FUNCTION_BLOCK FB_EventBuffer
VAR_INPUT
END_VAR

VAR_OUTPUT
    bError          : BOOL;             // Error flag
    nErrorID        : UDINT;            // Error code
END_VAR

VAR
    // ====================================================
    // INTERNAL RING BUFFER INSTANCE
    // From Tc2_Utilities library
    // ====================================================
    fbMemRingBuffer : FB_MemRingBuffer;

    // Buffer memory reference (set via FB_init)
    pBufferMemory   : PVOID;
    nBufferSize     : UDINT;
    bInitialized    : BOOL := FALSE;

END_VAR
```

```
// ====================================================
// METHOD: AddEvent
// Wraps the A_AddTail action with a clean interface
// Returns TRUE if event was successfully added
// ====================================================

METHOD PUBLIC AddEvent : BOOL
VAR_INPUT
    stEvent         : REFERENCE TO ST_Event;    // Event to add
END_VAR
```

```

// Check initialization
IF NOT bInitialized THEN
    AddEvent := FALSE;
    bError := TRUE;
    nErrorID := 16#8001;    // Not initialized
    RETURN;
END_IF

// ====================================================
// STEP 1: Set the write pointer and size
// FB_MemRingBuffer uses internal variables, not method params
// pWrite = pointer to the data we want to store
// cbWrite = size of that data in bytes
// ====================================================
fbMemRingBuffer.pWrite := ADR(stEvent);
fbMemRingBuffer.cbWrite := SIZEOF(ST_Event);

// ====================================================
// STEP 2: Call the A_AddTail action
// This is the legacy "action" syntax from TwinCAT 2
// The action uses the pWrite and cbWrite values we just set
// ====================================================
fbMemRingBuffer.A_AddTail();

// ====================================================
// STEP 3: Check the result
// bOk indicates whether the action succeeded
// It will be FALSE if buffer is full
// ====================================================
IF fbMemRingBuffer.bOk THEN
    AddEvent := TRUE;
    bError := FALSE;
    nErrorID := 0;
ELSE
    AddEvent := FALSE;
    bError := TRUE;
    nErrorID := 16#8002;    // Buffer full or write error
END_IF
```

```
// ====================================================
// METHOD: GetAndRemoveEvent
// Wraps the A_RemoveHead action
// Returns TRUE if an event was retrieved
// ====================================================

METHOD PUBLIC GetAndRemoveEvent : BOOL
VAR_OUTPUT
    stEvent         : ST_Event;     // Retrieved event
END_VAR

// Check initialization
IF NOT bInitialized THEN
    GetAndRemoveEvent := FALSE;
    RETURN;
END_IF

// Check if buffer has entries
```

```
IF fbMemRingBuffer.nCount = 0 THEN
    GetAndRemoveEvent := FALSE;
    RETURN;
END_IF

// ====================================================
// Set read pointer and size, then call action
// ====================================================
fbMemRingBuffer.pRead := ADR(stEvent);
fbMemRingBuffer.cbRead := SIZEOF(ST_Event);

// Call the remove action
fbMemRingBuffer.A_RemoveHead();

// Return result
GetAndRemoveEvent := fbMemRingBuffer.bOk;
```

```
// ====================================================
// METHOD: GetNumberOfEvents
// Returns the current count of events in the buffer
// ====================================================

METHOD PUBLIC GetNumberOfEvents : UDINT

IF NOT bInitialized THEN
    GetNumberOfEvents := 0;
    RETURN;
END_IF

// nCount is automatically maintained by FB_MemRingBuffer
GetNumberOfEvents := fbMemRingBuffer.nCount;
```

Warning: The `FB_MemRingBuffer` uses **Actions** rather than Methods. You must set the internal variables (`pWrite`, `cbWrite`, etc.) **before** calling the action. The action reads these variables internally—there are no parameters passed directly to the action call.

3 The "8-Byte Discovery" (Debugging with TDD)

During the **Red-Green-Refactor** cycle, tests initially failed even when the buffer appeared large enough to hold the events.

- **The Issue:** Because the source code for Beckhoff libraries is not public, the internal management of the buffer is a "black box".
- **The Finding:** The `FB_MemRingBuffer` requires a small amount of **internal overhead space** for every data set added to the buffer.
- **The Solution:** To make the tests pass, the developer added **8 extra bytes** of buffer space for every potential event in the array.

Debug Discovery: When tests failed with "buffer full" errors despite correct size calculations, investigation revealed that `FB_MemRingBuffer` uses approximately 8 bytes

of overhead per entry for internal pointer management. This is undocumented behavior discovered through TDD.

Calculating Buffer Storage

```
// ====================================================
// BUFFER SIZE CALCULATION WITH OVERHEAD
// ====================================================
// The FB_MemRingBuffer requires internal overhead space
// for managing each data set stored in the buffer.
// Through testing, this overhead was found to be ~8 bytes
// per entry.
//
// Formula:
// Total Size = MAX_EVENTS * (SIZEOF(ST_Event) + OVERHEAD)
// where OVERHEAD = 8 bytes per entry
// ====================================================

VAR CONSTANT
    // Maximum number of events to store
    MAX_EVENTS              : UDINT := 100;

    // Size of one event structure
    EVENT_SIZE              : UDINT := SIZEOF(ST_Event);

    // ====================================================
    // OVERHEAD BYTES PER ENTRY
    // FB_MemRingBuffer uses internal pointers/headers
    // for each data set. This overhead is NOT documented
    // but was discovered during TDD testing.
    // Without this overhead, the buffer reports "full"
    // before reaching the expected capacity.
    // ====================================================
    OVERHEAD_PER_ENTRY      : UDINT := 8;

    // Total buffer size calculation
    // Each event needs: its own size + management overhead
    BUFFER_SIZE             : UDINT := MAX_EVENTS * (EVENT_SIZE + OVERHEAD_PER_ENTRY);

END_VAR

VAR
    // ====================================================
    // BUFFER MEMORY ALLOCATION
    // Size includes overhead for FB_MemRingBuffer internals
    // Example: 100 events * (128 bytes + 8 bytes) = 13,600 bytes
    // ====================================================
    aEventBufferMemory      : ARRAY[0..(MAX_EVENTS * (SIZEOF(ST_Event) + 8)) - 1] OF BYTE;

    // Alternative: Use calculated constant (if compiler supports)
    // aEventBufferMemory    : ARRAY[0..BUFFER_SIZE - 1] OF BYTE;

    // Event buffer instance
    fbEventBuffer           : FB_EventBuffer;
```

```
END_VAR
```

```
// ====================================================
// PRACTICAL EXAMPLE: Buffer sizing for different capacities
// ====================================================

VAR CONSTANT
    // Small buffer for testing (10 events)
    SMALL_BUFFER_EVENTS      : UDINT := 10;
    SMALL_BUFFER_SIZE        : UDINT := SMALL_BUFFER_EVENTS * (SIZEOF(ST_Event) + 8);

    // Medium buffer for typical use (100 events)
    MEDIUM_BUFFER_EVENTS     : UDINT := 100;
    MEDIUM_BUFFER_SIZE       : UDINT := MEDIUM_BUFFER_EVENTS * (SIZEOF(ST_Event) + 8);

    // Large buffer for high-frequency logging (1000 events)
    LARGE_BUFFER_EVENTS      : UDINT := 1000;
    LARGE_BUFFER_SIZE        : UDINT := LARGE_BUFFER_EVENTS * (SIZEOF(ST_Event) + 8);

END_VAR

VAR
    // Allocate appropriately sized buffers
    // +8 bytes overhead per entry for FB_MemRingBuffer management
    aSmallBuffer     : ARRAY[0..1359] OF BYTE;    // ~10 * (128+8)
    aMediumBuffer    : ARRAY[0..13599] OF BYTE;   // ~100 * (128+8)
    aLargeBuffer     : ARRAY[0..135999] OF BYTE; // ~1000 * (128+8)

END_VAR

// ====================================================
// WHY +8 BYTES?
// ====================================================
// The FB_MemRingBuffer internally maintains:
// - Pointer to next entry (4 bytes on 32-bit systems)
// - Length of data set (4 bytes)
// Total: 8 bytes of overhead per stored data set
//
// This allows the ring buffer to:
// - Handle variable-length data sets
// - Navigate through the buffer efficiently
// - Know where each entry starts and ends
// ====================================================
```

Events	Event Size	Overhead	Total Buffer Size
10	128 bytes	$8 \times 10 = 80$	1,360 bytes
100	128 bytes	$8 \times 100 = 800$	13,600 bytes
1000	128 bytes	$8 \times 1000 = 8{,}000$	136,000 bytes

Table 3: Buffer Size Calculations Including Overhead

Tip: When working with closed-source library function blocks, TDD helps discover undocumented behaviors like this overhead requirement. Always verify capacity with tests that push the buffer to its limits.

4 Integrating the Buffer via Dependency Injection

To maintain a modular design, the **CSV Persistent Event Storage** is updated to use the new buffer.

- **Reference Injection:** Instead of the storage logger owning the buffer, it receives a **REFERENCE TO** an event buffer via its `FB_init` constructor.
- **Decoupled Logic:** The logger no longer waits for a single event; instead, it cyclically checks if the buffer's `nCount` is greater than zero. If entries exist, it retrieves them one by one and writes them to the CSV file.

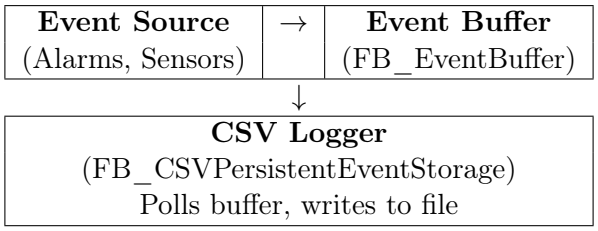

Figure 2: Decoupled Event Logging Architecture

Updated Logger Logic

```
// ====================================================
// FUNCTION BLOCK: FB_CSVPersistentEventStorage
// Writes events from buffer to CSV file
// Uses dependency injection for the event buffer
// ====================================================

FUNCTION_BLOCK FB_CSVPersistentEventStorage
VAR_INPUT
    sFilePath        : STRING(255);       // Path to CSV file
    bEnable          : BOOL := TRUE;      // Enable logging
END_VAR

VAR_OUTPUT
    bBusy            : BOOL;              // Currently writing
    bError           : BOOL;              // Error occurred
    nEventsWritten   : UDINT;             // Total events written
END_VAR

VAR
    // ====================================================
    // INJECTED DEPENDENCY
    // Reference to external event buffer
    // Set via FB_init constructor
    // ====================================================
    refEventBuffer   : REFERENCE TO FB_EventBuffer;
```

```
    // State machine
    eState          : E_LoggerState := E_LoggerState.IDLE;

    // Current event being processed
    stCurrentEvent  : ST_Event;

    // File handling
    fbFileOpen      : FB_FileOpen;
    fbFileWrite     : FB_FileWrite;
    fbFileClose     : FB_FileClose;
    hFile           : UINT;             // File handle

    // CSV formatting
    sLineBuffer     : STRING(500);

END_VAR
```

```
// ====================================================
// METHOD: FB_init (Constructor with Dependency Injection)
// Receives reference to external event buffer
// ====================================================

METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains    : BOOL;
    bInCopyCode     : BOOL;

    // Injected dependency: reference to event buffer
    refBuffer       : REFERENCE TO FB_EventBuffer;
END_VAR

// Store reference to the event buffer
refEventBuffer REF= refBuffer;

// Validate the reference
IF NOT __ISVALIDREF(refEventBuffer) THEN
    bError := TRUE;
    FB_init := FALSE;
    RETURN;
END_IF

FB_init := TRUE;
```

```
// ====================================================
// MAIN STATE MACHINE - Body of FB_CSVPersistentEventStorage
// Cyclically checks buffer and processes events
// ====================================================

// Only process if enabled
IF NOT bEnable THEN
    eState := E_LoggerState.IDLE;
    RETURN;
END_IF

CASE eState OF

    // ====================================================
```

```
    // STATE: IDLE / WAIT_FOR_EVENT
    // Continuously poll the buffer for new events
    // This replaces the old "wait for single event" logic
    // ====================================================
    E_LoggerState.IDLE,
    E_LoggerState.WAIT_FOR_EVENT:

        bBusy := FALSE;

        // ====================================================
        // CHECK IF BUFFER HAS EVENTS
        // GetNumberOfEvents() returns the current count
        // If > 0, we have work to do
        // ====================================================
        IF refEventBuffer.GetNumberOfEvents() > 0 THEN
            // Events are waiting - start processing
            eState := E_LoggerState.GET_EVENT;
        END_IF

    // ====================================================
    // STATE: GET_EVENT
    // Retrieve the oldest event from the buffer
    // ====================================================
    E_LoggerState.GET_EVENT:

        bBusy := TRUE;

        // Get and remove the oldest event (FIFO)
        IF refEventBuffer.GetAndRemoveEvent(stEvent => stCurrentEvent
    ) THEN
            // Successfully retrieved event
            eState := E_LoggerState.OPEN_FILE;
        ELSE
            // Buffer became empty (race condition) or error
            eState := E_LoggerState.WAIT_FOR_EVENT;
        END_IF

    // ====================================================
    // STATE: OPEN_FILE
    // Open the CSV file for appending
    // ====================================================
    E_LoggerState.OPEN_FILE:

        fbFileOpen(
            sPathName := sFilePath,
            nMode := FOPEN_MODEAPPEND OR FOPEN_MODETEXT,
            bExecute := TRUE
        );

        IF NOT fbFileOpen.bBusy THEN
            fbFileOpen(bExecute := FALSE);

            IF NOT fbFileOpen.bError THEN
                hFile := fbFileOpen.hFile;
                eState := E_LoggerState.FORMAT_DATA;
            ELSE
                bError := TRUE;
                eState := E_LoggerState.ERROR;
```

```
            END_IF
        END_IF

    // ====================================================
    // STATE: FORMAT_DATA
    // Convert event to CSV line format
    // ====================================================
    E_LoggerState.FORMAT_DATA:

        // Format: Timestamp,EventID,Severity,Description
        sLineBuffer := CONCAT(
            DT_TO_STRING(stCurrentEvent.tTimestamp), ',');
        sLineBuffer := CONCAT(sLineBuffer,
            DINT_TO_STRING(stCurrentEvent.nEventID));
        sLineBuffer := CONCAT(sLineBuffer, ',');
        sLineBuffer := CONCAT(sLineBuffer,
            INT_TO_STRING(stCurrentEvent.nSeverity));
        sLineBuffer := CONCAT(sLineBuffer, ',');
        sLineBuffer := CONCAT(sLineBuffer,
            stCurrentEvent.sDescription);
        sLineBuffer := CONCAT(sLineBuffer, '$N');    // Newline

        eState := E_LoggerState.WRITE_DATA;

    // ====================================================
    // STATE: WRITE_DATA
    // Write the formatted line to file
    // ====================================================
    E_LoggerState.WRITE_DATA:

        fbFileWrite(
            hFile := hFile,
            pWriteBuff := ADR(sLineBuffer),
            cbWriteLen := LEN(sLineBuffer),
            bExecute := TRUE
        );

        IF NOT fbFileWrite.bBusy THEN
            fbFileWrite(bExecute := FALSE);

            IF NOT fbFileWrite.bError THEN
                nEventsWritten := nEventsWritten + 1;
                eState := E_LoggerState.CLOSE_FILE;
            ELSE
                bError := TRUE;
                eState := E_LoggerState.ERROR;
            END_IF
        END_IF

    // ====================================================
    // STATE: CLOSE_FILE
    // Close file and check for more events
    // ====================================================
    E_LoggerState.CLOSE_FILE:

        fbFileClose(
            hFile := hFile,
            bExecute := TRUE
```

```
        );

        IF NOT fbFileClose.bBusy THEN
            fbFileClose(bExecute := FALSE);

            // Return to waiting state
            // Will immediately check for more events
            eState := E_LoggerState.WAIT_FOR_EVENT;
        END_IF

    // ====================================================
    // STATE: ERROR
    // Handle errors - could add retry logic here
    // ====================================================
    E_LoggerState.ERROR:

        bBusy := FALSE;
        // Stay in error state until reset
        // Could add automatic retry after timeout

END_CASE
```

```
// ====================================================
// ENUMERATION: E_LoggerState
// States for the CSV logger state machine
// ====================================================

TYPE E_LoggerState :
(
    IDLE := 0,
    WAIT_FOR_EVENT := 1,
    GET_EVENT := 2,
    OPEN_FILE := 3,
    FORMAT_DATA := 4,
    WRITE_DATA := 5,
    CLOSE_FILE := 6,
    ERROR := 99
);
END_TYPE
```

```
// ====================================================
// USAGE EXAMPLE: Main program with dependency injection
// ====================================================

PROGRAM MAIN
VAR
    // Buffer memory with overhead
    aBufferMemory    : ARRAY[0..13599] OF BYTE;  // 100 events

    // Event buffer instance
    fbEventBuffer    : FB_EventBuffer(
                         pBufferMemory := ADR(aBufferMemory),
                         nBufferMemorySize := SIZEOF(aBufferMemory)
                       );

    // CSV logger with injected buffer reference
    fbCSVLogger      : FB_CSVPersistentEventStorage(
                         refBuffer := fbEventBuffer
```

```
                        );

    // Test event
    stTestEvent      : ST_Event;
    bAddEvent        : BOOL;

END_VAR

// Configure logger
fbCSVLogger.sFilePath := 'C:\Logs\events.csv';
fbCSVLogger.bEnable := TRUE;

// Call logger cyclically - it will check buffer automatically
fbCSVLogger();

// Add events from various sources
IF bAddEvent THEN
    stTestEvent.nEventID := 1001;
    stTestEvent.sDescription := 'Production started';
    stTestEvent.tTimestamp := DT#2024-01-15-10:30:00;
    fbEventBuffer.AddEvent(stEvent := stTestEvent);
    bAddEvent := FALSE;
END_IF
```

Note: Dependency injection makes the logger testable in isolation. During testing, you can inject a mock buffer to verify the logger's behavior without actual file I/O operations.

5 The Professional Value of TDD

Test-Driven Development is as much about **design** as it is about testing.

- **Live Documentation:** Test cases (e.g., "Add four events, expect all to be added except the last") act as documentation that is guaranteed to be up-to-date; if the code changes and the documentation (tests) doesn't, the build fails.
- **Regression Testing:** Developers can make bold changes to a large codebase and rerun tests instantly to ensure nothing has broken.
- **Future-Proofing:** Most of a developer's time is spent understanding **existing code**. TDD provides a roadmap for the next developer to understand the intended behavior of the software.
- **Continuous Integration (CI):** These tests can be executed automatically on a build server whenever code is committed to a repository like Git.

Tip: Think of tests as executable specifications. When a new team member asks "what should this function block do?", point them to the test suite. The tests describe every expected behavior in code that is verified to be accurate.

Quick Reference Card

FB_MemRingBuffer Actions

TDD Benefit	Practical Impact
Live Documentation	Tests describe expected behavior
Regression Testing	Safe refactoring and updates
Design Feedback	Forces modular, testable code
Bug Discovery	Found 8-byte overhead issue early
CI Integration	Automated quality gates

Table 4: TDD Benefits Demonstrated in This Implementation

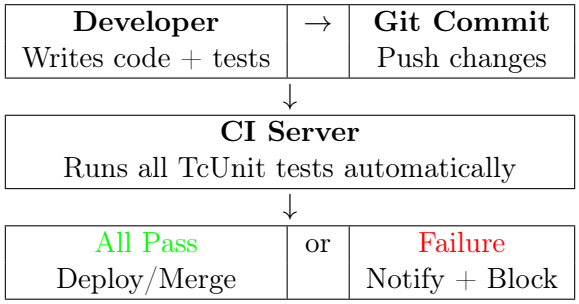

Figure 3: TDD in Continuous Integration Workflow

```
// Add data to buffer
fbMemRingBuffer.pWrite := ADR(stData);
fbMemRingBuffer.cbWrite := SIZEOF(stData);
fbMemRingBuffer.A_AddTail();
bSuccess := fbMemRingBuffer.bOk;

// Read and remove oldest data
fbMemRingBuffer.pRead := ADR(stData);
fbMemRingBuffer.cbRead := SIZEOF(stData);
fbMemRingBuffer.A_RemoveHead();
bSuccess := fbMemRingBuffer.bOk;

// Check count
nCount := fbMemRingBuffer.nCount;
```

Buffer Size Formula

```
// Required buffer size with overhead
BufferSize = MAX_EVENTS * (SIZEOF(ST_Event) + 8)

// Example: 100 events of 128 bytes each
BufferSize = 100 * (128 + 8) = 13,600 bytes
```

Dependency Injection Pattern

```
// In FB_init constructor
METHOD FB_init : BOOL
VAR_INPUT
    refDependency : REFERENCE TO FB_SomeType;
END_VAR
    refInternal REF= refDependency;
END_METHOD
```

```

// At instantiation
fbConsumer : FB_Consumer(refDependency := fbProvider);
```

Key Discoveries from TDD

- FB_MemRingBuffer needs +8 bytes overhead per entry
- Actions set variables before call (not parameters)
- Dependency injection enables testable design
- Tests catch undocumented library behaviors

Last Updated: March 17, 2026